

Node/TypeScript MCP Server Implementation Guide

Overview

This document provides Node/TypeScript-specific best practices and examples for implementing MCP servers using the MCP TypeScript SDK. It covers project structure, server setup, tool registration patterns, input validation with Zod, error handling, and complete working examples.

Quick Reference

Key Imports

```
typescript
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StreamableHTTPServerTransport } from
"@modelcontextprotocol/sdk/server/streamableHttp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import express from "express";
import { z } from "zod";
```

Server Initialization

```
typescript
const server = new McpServer({
  name: "service-mcp-server",
  version: "1.0.0"
});
```

Tool Registration Pattern

```
typescript
server.registerTool(
  "tool_name",
  {
    title: "Tool Display Name",
    description: "What the tool does",
    inputSchema: { param: z.string() },
    outputSchema: { result: z.string() }
```

```
},
async ({ param }) => {
  const output = { result: `Processed: ${param}` };
  return {
    content: [{ type: "text", text: JSON.stringify(output) }],
    structuredContent: output // Modern pattern for structured data
  };
}
);
```

MCP TypeScript SDK

The official MCP TypeScript SDK provides:

- `McpServer` class for server initialization
- `registerTool` method for tool registration
- Zod schema integration for runtime input validation
- Type-safe tool handler implementations

IMPORTANT - Use Modern APIs Only:

- **DO use:** `server.registerTool()`, `server.registerResource()`, `server.registerPrompt()`
- **DO NOT use:** Old deprecated APIs such as `server.tool()`, `server.setRequestHandler(ListToolsRequestSchema, ...)`, or manual handler registration
- The `register*` methods provide better type safety, automatic schema handling, and are the recommended approach

See the MCP SDK documentation in the references for complete details.

Server Naming Convention

Node/TypeScript MCP servers must follow this naming pattern:

- **Format:** `{service}-mcp-server` (lowercase with hyphens)
- **Examples:** `github-mcp-server`, `jira-mcp-server`, `stripe-mcp-server`

The name should be:

- General (not tied to specific features)
- Descriptive of the service/API being integrated

- Easy to infer from the task description
- Without version numbers or dates

Project Structure

Create the following structure for Node/TypeScript MCP servers:

```
{service}-mcp-server/
├── package.json
├── tsconfig.json
├── README.md
├── src/
│   ├── index.ts      # Main entry point with McpServer initialization
│   ├── types.ts      # TypeScript type definitions and interfaces
│   ├── tools/        # Tool implementations (one file per domain)
│   ├── services/     # API clients and shared utilities
│   ├── schemas/      # Zod validation schemas
│   └── constants.ts  # Shared constants (API_URL, CHARACTER_LIMIT, etc.)
└── dist/             # Built JavaScript files (entry point: dist/index.js)
```

Tool Implementation

Tool Naming

Use snake_case for tool names (e.g., "search_users", "create_project", "get_channel_info") with clear, action-oriented names.

Avoid Naming Conflicts: Include the service context to prevent overlaps:

- Use "slack_send_message" instead of just "send_message"
- Use "github_create_issue" instead of just "create_issue"
- Use "asana_list_tasks" instead of just "list_tasks"

Tool Structure

Tools are registered using the `registerTool` method with the following requirements:

- Use Zod schemas for runtime input validation and type safety
- The `description` field must be explicitly provided - JSDoc comments are NOT automatically extracted
- Explicitly provide `title`, `description`, `inputSchema`, and `annotations`
- The `inputSchema` must be a Zod schema object (not a JSON schema)
- Type all parameters and return values explicitly

typescript

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { z } from "zod";

const server = new McpServer({
  name: "example-mcp",
  version: "1.0.0"
});

// Zod schema for input validation
const UserSearchInputSchema = z.object({
  query: z.string()
    .min(2, "Query must be at least 2 characters")
    .max(200, "Query must not exceed 200 characters")
    .describe("Search string to match against names/emails"),
  limit: z.number()
    .int()
    .min(1)
    .max(100)
    .default(20)
    .describe("Maximum results to return"),
  offset: z.number()
    .int()
    .min(0)
    .default(0)
    .describe("Number of results to skip for pagination"),
  response_format: z.nativeEnum(ResponseFormat)
    .default(ResponseFormat.MARKDOWN)
    .describe("Output format: 'markdown' for human-readable or 'json' for machine-readable")
}).strict();

// Type definition from Zod schema
type UserSearchInput = z.infer<typeof UserSearchInputSchema>;

server.registerTool(
  "example_search_users",
  {
    title: "Search Example Users",
    description: `Search for users in the Example system by name, email, or team.
```

This tool searches across all user profiles in the Example platform, supporting partial matches and various search filters. It does NOT create or modify users, only searches existing ones.

Args:

- query (string): Search string to match against names/emails
- limit (number): Maximum results to return, between 1-100 (default: 20)
- offset (number): Number of results to skip for pagination (default: 0)
- response_format ('markdown' | 'json'): Output format (default: 'markdown')

Returns:

For JSON format: Structured data with schema:

```
{
  "total": number,      // Total number of matches found
  "count": number,      // Number of results in this response
  "offset": number,     // Current pagination offset
  "users": [
    {
      "id": string,      // User ID (e.g., "U123456789")
      "name": string,    // Full name (e.g., "John Doe")
      "email": string,   // Email address
      "team": string,    // Team name (optional)
      "active": boolean  // Whether user is active
    }
  ],
  "has_more": boolean,  // Whether more results are available
  "next_offset": number // Offset for next page (if has_more is true)
}
```

Examples:

- Use when: "Find all marketing team members" -> params with query="team:marketing"
- Use when: "Search for John's account" -> params with query="john"
- Don't use when: You need to create a user (use example_create_user instead)

Error Handling:

- Returns "Error: Rate limit exceeded" if too many requests (429 status)
- Returns "No users found matching '<query>'" if search returns empty`

```
inputSchema: UserSearchInputSchema,
annotations: {
  readOnlyHint: true,
  destructiveHint: false,
  idempotentHint: true,
  openWorldHint: true
},
},
async (params: UserSearchInput) => {
  try {
    // Input validation is handled by Zod schema
    // Make API request using validated parameters
```

```

const data = await makeApiRequest<any>(
  "users/search",
  "GET",
  undefined,
  {
    q: params.query,
    limit: params.limit,
    offset: params.offset
  }
);

const users = data.users || [];
const total = data.total || 0;

if (!users.length) {
  return {
    content: [{
      type: "text",
      text: `No users found matching '${params.query}'`
    }]
  };
}

// Prepare structured output
const output = {
  total,
  count: users.length,
  offset: params.offset,
  users: users.map((user: any) => ({
    id: user.id,
    name: user.name,
    email: user.email,
    ...(user.team ? { team: user.team } : {}),
    active: user.active ?? true
  })),
  has_more: total > params.offset + users.length,
  ...(total > params.offset + users.length ? {
    next_offset: params.offset + users.length
  } : {})
};

// Format text representation based on requested format
let textContent: string;
if (params.response_format === ResponseFormat.MARKDOWN) {

```

```

const lines = [`# User Search Results: '${params.query}'`, '',
  `Found ${total} users (showing ${users.length})`, ''];
for (const user of users) {
  lines.push(`## ${user.name} (${user.id})`);
  lines.push(`- **Email**: ${user.email}`);
  if (user.team) lines.push(`- **Team**: ${user.team}`);
  lines.push('');
}
textContent = lines.join("\n");
} else {
  textContent = JSON.stringify(output, null, 2);
}

return {
  content: [{ type: "text", text: textContent }],
  structuredContent: output // Modern pattern for structured data
};
} catch (error) {
  return {
    content: [{
      type: "text",
      text: handleApiError(error)
    }]
  };
}
}
);

```

Zod Schemas for Input Validation

Zod provides runtime type validation:

typescript

```

import { z } from "zod";

// Basic schema with validation
const CreateUserSchema = z.object({
  name: z.string()
    .min(1, "Name is required")
    .max(100, "Name must not exceed 100 characters"),
  email: z.string()
    .email("Invalid email format"),
  age: z.number()
    .int("Age must be a whole number")

```

```

    .min(0, "Age cannot be negative")
    .max(150, "Age cannot be greater than 150")
  }).strict(); // Use .strict() to forbid extra fields

// Enums
enum ResponseFormat {
  MARKDOWN = "markdown",
  JSON = "json"
}

const SearchSchema = z.object({
  response_format: z.nativeEnum(ResponseFormat)
    .default(ResponseFormat.MARKDOWN)
    .describe("Output format")
});

// Optional fields with defaults
const PaginationSchema = z.object({
  limit: z.number()
    .int()
    .min(1)
    .max(100)
    .default(20)
    .describe("Maximum results to return"),
  offset: z.number()
    .int()
    .min(0)
    .default(0)
    .describe("Number of results to skip")
});

```

Response Format Options

Support multiple output formats for flexibility:

```

typescript
enum ResponseFormat {
  MARKDOWN = "markdown",
  JSON = "json"
}

const inputSchema = z.object({
  query: z.string(),
  response_format: z.nativeEnum(ResponseFormat)

```



```
.default(ResponseFormat.MARKDOWN)
.describe("Output format: 'markdown' for human-readable or 'json' for machine-readable")
});
```

Markdown format:

- Use headers, lists, and formatting for clarity
- Convert timestamps to human-readable format
- Show display names with IDs in parentheses
- Omit verbose metadata
- Group related information logically

JSON format:

- Return complete, structured data suitable for programmatic processing
- Include all available fields and metadata
- Use consistent field names and types

Pagination Implementation

For tools that list resources:

typescript

```
const ListSchema = z.object({
  limit: z.number().int().min(1).max(100).default(20),
  offset: z.number().int().min(0).default(0)
});

async function listItems(params: z.infer<typeof ListSchema>) {
  const data = await apiRequest(params.limit, params.offset);

  const response = {
    total: data.total,
    count: data.items.length,
    offset: params.offset,
    items: data.items,
    has_more: data.total > params.offset + data.items.length,
    next_offset: data.total > params.offset + data.items.length
      ? params.offset + data.items.length
      : undefined
  };

  return JSON.stringify(response, null, 2);
}
```

Character Limits and Truncation

Add a CHARACTER_LIMIT constant to prevent overwhelming responses:

typescript

// At module level in constants.ts

```
export const CHARACTER_LIMIT = 25000; // Maximum response size in characters
```

```
async function searchTool(params: SearchInput) {  
  let result = generateResponse(data);  
  
  // Check character limit and truncate if needed  
  if (result.length > CHARACTER_LIMIT) {  
    const truncatedData = data.slice(0, Math.max(1, data.length / 2));  
    response.data = truncatedData;  
    response.truncated = true;  
    response.truncation_message =  
      `Response truncated from ${data.length} to ${truncatedData.length} items. ` +  
      `Use 'offset' parameter or add filters to see more results.`;  
    result = JSON.stringify(response, null, 2);  
  }  
  
  return result;  
}
```

Error Handling

Provide clear, actionable error messages:

typescript

```
import axios, { AxiosError } from "axios";
```

```
function handleApiError(error: unknown): string {  
  if (error instanceof AxiosError) {  
    if (error.response) {  
      switch (error.response.status) {  
        case 404:  
          return "Error: Resource not found. Please check the ID is correct.";  
        case 403:  
          return "Error: Permission denied. You don't have access to this resource.";  
        case 429:  
          return "Error: Rate limit exceeded. Please wait before making more requests.";  
        default:  
          return `Error: API request failed with status ${error.response.status}`;  
      }  
    }  
  }  
}
```

```

    }
  } else if (error.code === "ECONNABORTED") {
    return "Error: Request timed out. Please try again.";
  }
}
return `Error: Unexpected error occurred: ${error instanceof Error ? error.message :
String(error)}`;
}

```

Shared Utilities

Extract common functionality into reusable functions:

```

typescript
// Shared API request function
async function makeApiRequest<T>(
  endpoint: string,
  method: "GET" | "POST" | "PUT" | "DELETE" = "GET",
  data?: any,
  params?: any
): Promise<T> {
  try {
    const response = await axios({
      method,
      url: `${API_BASE_URL}/${endpoint}`,
      data,
      params,
      timeout: 30000,
      headers: {
        "Content-Type": "application/json",
        "Accept": "application/json"
      }
    });
    return response.data;
  } catch (error) {
    throw error;
  }
}

```

Async/Await Best Practices

Always use async/await for network requests and I/O operations:

typescript

// Good: Async network request

```
async function fetchData(resourceId: string): Promise<ResourceData> {  
  const response = await axios.get(`${API_URL}/resource/${resourceId}`);  
  return response.data;  
}
```

// Bad: Promise chains

```
function fetchData(resourceId: string): Promise<ResourceData> {  
  return axios.get(`${API_URL}/resource/${resourceId}`)  
    .then(response => response.data); // Harder to read and maintain  
}
```

TypeScript Best Practices

1. **Use Strict TypeScript:** Enable strict mode in tsconfig.json
2. **Define Interfaces:** Create clear interface definitions for all data structures
3. **Avoid `any`:** Use proper types or `unknown` instead of `any`
4. **Zod for Runtime Validation:** Use Zod schemas to validate external data
5. **Type Guards:** Create type guard functions for complex type checking
6. **Error Handling:** Always use try-catch with proper error type checking
7. **Null Safety:** Use optional chaining (`?.`) and nullish coalescing (`??`)

typescript

// Good: Type-safe with Zod and interfaces

```
interface UserResponse {  
  id: string;  
  name: string;  
  email: string;  
  team?: string;  
  active: boolean;  
}
```

```
const UserSchema = z.object({  
  id: z.string(),  
  name: z.string(),  
  email: z.string().email(),  
  team: z.string().optional(),  
  active: z.boolean()  
});
```

```
type User = z.infer<typeof UserSchema>;
```

```

async function getUser(id: string): Promise<User> {
  const data = await apiCall(`/users/${id}`);
  return UserSchema.parse(data); // Runtime validation
}

```

// Bad: Using any

```

async function getUser(id: string): Promise<any> {
  return await apiCall(`/users/${id}`); // No type safety
}

```

Package Configuration

package.json

json

```

{
  "name": "{service}-mcp-server",
  "version": "1.0.0",
  "description": "MCP server for {Service} API integration",
  "type": "module",
  "main": "dist/index.js",
  "scripts": {
    "start": "node dist/index.js",
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "clean": "rm -rf dist"
  },
  "engines": {
    "node": ">=18"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "^1.6.1",
    "axios": "^1.7.9",
    "zod": "^3.23.8"
  },
  "devDependencies": {
    "@types/node": "^22.10.0",
    "tsx": "^4.19.2",
    "typescript": "^5.7.2"
  }
}

```

tsconfig.json

json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "Node16",
    "moduleResolution": "Node16",
    "lib": ["ES2022"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,
    "allowSyntheticDefaultImports": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Complete Example

typescript

```
#!/usr/bin/env node
```

```
/**
```

```
 * MCP Server for Example Service.
```

```
 *
```

```
 * This server provides tools to interact with Example API, including user search,
```

```
 * project management, and data export capabilities.
```

```
 */
```

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
```

```
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
```

```
import { z } from "zod";
```

```
import axios, { AxiosError } from "axios";
```

```
// Constants
```

```
const API_BASE_URL = "https://api.example.com/v1";
```

```
const CHARACTER_LIMIT = 25000;
```

```
// Enums
```

```
enum ResponseFormat {
```

```
  MARKDOWN = "markdown",
```

```

    JSON = "json"
  }

// Zod schemas
const UserSearchInputSchema = z.object({
  query: z.string()
    .min(2, "Query must be at least 2 characters")
    .max(200, "Query must not exceed 200 characters")
    .describe("Search string to match against names/emails"),
  limit: z.number()
    .int()
    .min(1)
    .max(100)
    .default(20)
    .describe("Maximum results to return"),
  offset: z.number()
    .int()
    .min(0)
    .default(0)
    .describe("Number of results to skip for pagination"),
  response_format: z.nativeEnum(ResponseFormat)
    .default(ResponseFormat.MARKDOWN)
    .describe("Output format: 'markdown' for human-readable or 'json' for machine-readable")
}).strict();

type UserSearchInput = z.infer<typeof UserSearchInputSchema>;

// Shared utility functions
async function makeApiRequest<T>({
  endpoint: string,
  method: "GET" | "POST" | "PUT" | "DELETE" = "GET",
  data?: any,
  params?: any
}): Promise<T> {
  try {
    const response = await axios({
      method,
      url: `${API_BASE_URL}/${endpoint}`,
      data,
      params,
      timeout: 30000,
      headers: {
        "Content-Type": "application/json",
        "Accept": "application/json"
      }
    });
    return response.data;
  } catch (error) {
    console.error("API Request Error:", error);
    throw error;
  }
}

```

```

    }
  });
  return response.data;
} catch (error) {
  throw error;
}
}

```

```

function handleApiError(error: unknown): string {
  if (error instanceof AxiosError) {
    if (error.response) {
      switch (error.response.status) {
        case 404:
          return "Error: Resource not found. Please check the ID is correct.";
        case 403:
          return "Error: Permission denied. You don't have access to this resource.";
        case 429:
          return "Error: Rate limit exceeded. Please wait before making more requests.";
        default:
          return `Error: API request failed with status ${error.response.status}`;
      }
    } else if (error.code === "ECONNABORTED") {
      return "Error: Request timed out. Please try again.";
    }
  }
  return `Error: Unexpected error occurred: ${error instanceof Error ? error.message : String(error)}`;
}

```

```

// Create MCP server instance
const server = new McpServer({
  name: "example-mcp",
  version: "1.0.0"
});

```

```

// Register tools
server.registerTool(
  "example_search_users",
  {
    title: "Search Example Users",
    description: `[Full description as shown above]`,
    inputSchema: UserSearchInputSchema,
    annotations: {
      readOnlyHint: true,

```



```

        destructiveHint: false,
        idempotentHint: true,
        openWorldHint: true
    }
},
async (params: UserSearchInput) => {
    // Implementation as shown above
}
);

// Main function
// For stdio (local):
async function runStdio() {
    if (!process.env.EXAMPLE_API_KEY) {
        console.error("ERROR: EXAMPLE_API_KEY environment variable is required");
        process.exit(1);
    }

    const transport = new StdioServerTransport();
    await server.connect(transport);
    console.error("MCP server running via stdio");
}

// For streamable HTTP (remote):
async function runHTTP() {
    if (!process.env.EXAMPLE_API_KEY) {
        console.error("ERROR: EXAMPLE_API_KEY environment variable is required");
        process.exit(1);
    }

    const app = express();
    app.use(express.json());

    app.post('/mcp', async (req, res) => {
        const transport = new StreamableHTTPServerTransport({
            sessionIdGenerator: undefined,
            enableJsonResponse: true
        });
        res.on('close', () => transport.close());
        await server.connect(transport);
        await transport.handleRequest(req, res, req.body);
    });

    const port = parseInt(process.env.PORT || '3000');

```

```

app.listen(port, () => {
  console.error(`MCP server running on http://localhost:${port}/mcp`);
});
}

// Choose transport based on environment
const transport = process.env.TRANSPORT || 'stdio';
if (transport === 'http') {
  runHTTP().catch(error => {
    console.error("Server error:", error);
    process.exit(1);
  });
} else {
  runStdio().catch(error => {
    console.error("Server error:", error);
    process.exit(1);
  });
}

```

Advanced MCP Features

Resource Registration

Expose data as resources for efficient, URI-based access:

typescript

```

import { ResourceTemplate } from "@modelcontextprotocol/sdk/types.js";

// Register a resource with URI template
server.registerResource(
  {
    uri: "file://documents/{name}",
    name: "Document Resource",
    description: "Access documents by name",
    mimeType: "text/plain"
  },
  async (uri: string) => {
    // Extract parameter from URI
    const match = uri.match(/^file:VVdocumentsV(.+)\$/);
    if (!match) {
      throw new Error("Invalid URI format");
    }
  }
)

```

```

const documentName = match[1];
const content = await loadDocument(documentName);

return {
  contents: [{
    uri,
    mimeType: "text/plain",
    text: content
  }]
};
}
);

// List available resources dynamically
server.registerResourceList(async () => {
  const documents = await getAvailableDocuments();
  return {
    resources: documents.map(doc => ({
      uri: `file://documents/${doc.name}`,
      name: doc.name,
      mimeType: "text/plain",
      description: doc.description
    }))
  };
});

```

When to use Resources vs Tools:

- **Resources:** For data access with simple URI-based parameters
- **Tools:** For complex operations requiring validation and business logic
- **Resources:** When data is relatively static or template-based
- **Tools:** When operations have side effects or complex workflows

Transport Options

The TypeScript SDK supports two main transport mechanisms:

Streamable HTTP (Recommended for Remote Servers)

```

typescript
import { StreamableHTTPServerTransport } from
"@modelcontextprotocol/sdk/server/streamableHttp.js";
import express from "express";

const app = express();
app.use(express.json());

```

```

app.post('/mcp', async (req, res) => {
  // Create new transport for each request (stateless, prevents request ID collisions)
  const transport = new StreamableHTTPServerTransport({
    sessionIdGenerator: undefined,
    enableJsonResponse: true
  });

  res.on('close', () => transport.close());

  await server.connect(transport);
  await transport.handleRequest(req, res, req.body);
});

app.listen(3000);

```

stdio (For Local Integrations)

typescript

```

import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";

const transport = new StdioServerTransport();
await server.connect(transport);

```

Transport selection:

- **Streamable HTTP:** Web services, remote access, multiple clients
- **stdio:** Command-line tools, local development, subprocess integration

Notification Support

Notify clients when server state changes:

typescript

```

// Notify when tools list changes
server.notification({
  method: "notifications/tools/list_changed"
});

// Notify when resources change
server.notification({
  method: "notifications/resources/list_changed"
});

```

Use notifications sparingly - only when server capabilities genuinely change.

Code Best Practices

Code Composability and Reusability

Your implementation **MUST** prioritize composability and code reuse:

1. **Extract Common Functionality:**
 - Create reusable helper functions for operations used across multiple tools
 - Build shared API clients for HTTP requests instead of duplicating code
 - Centralize error handling logic in utility functions
 - Extract business logic into dedicated functions that can be composed
 - Extract shared markdown or JSON field selection & formatting functionality
2. **Avoid Duplication:**
 - NEVER copy-paste similar code between tools
 - If you find yourself writing similar logic twice, extract it into a function
 - Common operations like pagination, filtering, field selection, and formatting should be shared
 - Authentication/authorization logic should be centralized

Building and Running

Always build your TypeScript code before running:

bash

Build the project

`npm run build`

Run the server

`npm start`

Development with auto-reload

`npm run dev`

Always ensure `npm run build` completes successfully before considering the implementation complete.

Quality Checklist

Before finalizing your Node/TypeScript MCP server implementation, ensure:

Strategic Design

- Tools enable complete workflows, not just API endpoint wrappers
- Tool names reflect natural task subdivisions
- Response formats optimize for agent context efficiency
- Human-readable identifiers used where appropriate
- Error messages guide agents toward correct usage

Implementation Quality

- FOCUSED IMPLEMENTATION: Most important and valuable tools implemented
- All tools registered using `registerTool` with complete configuration
- All tools include `title`, `description`, `inputSchema`, and `annotations`
- Annotations correctly set (`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`)
- All tools use Zod schemas for runtime input validation with `.strict()` enforcement
- All Zod schemas have proper constraints and descriptive error messages
- All tools have comprehensive descriptions with explicit input/output types
- Descriptions include return value examples and complete schema documentation
- Error messages are clear, actionable, and educational

TypeScript Quality

- TypeScript interfaces are defined for all data structures
- Strict TypeScript is enabled in `tsconfig.json`
- No use of `any` type - use `unknown` or proper types instead
- All async functions have explicit `Promise<T>` return types
- Error handling uses proper type guards (e.g., `axios.isAxiosError`, `z.ZodError`)

Advanced Features (where applicable)

- Resources registered for appropriate data endpoints
- Appropriate transport configured (stdio or streamable HTTP)
- Notifications implemented for dynamic server capabilities
- Type-safe with SDK interfaces

Project Configuration

- `Package.json` includes all necessary dependencies
- Build script produces working JavaScript in `dist/` directory
- Main entry point is properly configured as `dist/index.js`
- Server name follows format: `{service}-mcp-server`
- `tsconfig.json` properly configured with strict mode

Code Quality

- Pagination is properly implemented where applicable
- Large responses check CHARACTER_LIMIT constant and truncate with clear messages
- Filtering options are provided for potentially large result sets
- All network operations handle timeouts and connection errors gracefully
- Common functionality is extracted into reusable functions
- Return types are consistent across similar operations

Testing and Build

- `npm run build` completes successfully without errors
- `dist/index.js` created and executable
- Server runs: `node dist/index.js --help`
- All imports resolve correctly
- Sample tool calls work as expected